

Chapter 8 Multidimensional Arrays



Objectives

- ❑ To give examples of representing data using two-dimensional arrays (§8.1).
- ❑ To **declare variables** for two-dimensional arrays, **create arrays**, and **access array** elements in a two-dimensional array using row and column indexes (§8.2).
- ❑ To program common operations for two-dimensional arrays (**displaying arrays, summing all elements, finding the minimum and maximum elements, and random shuffling**) (§8.3).
- ❑ To **pass** two-dimensional arrays to methods (§8.4).
- ❑ To write a program for **grading multiple-choice questions** using two-dimensional arrays (§8.5).
- ❑ To use **multidimensional arrays** (§8.8).



Motivation

Thus far, you have used one-dimensional arrays to model linear collections of elements. You can use a two-dimensional array to represent a matrix or a table. For example, the following table that describes the distances between the cities can be represented using a two-dimensional array.

Distance Table (in miles)

	Chicago	Boston	New York	Atlanta	Miami	Dallas	Houston
Chicago	0	983	787	714	1375	967	1087
Boston	983	0	214	1102	1763	1723	1842
New York	787	214	0	888	1549	1548	1627
Atlanta	714	1102	888	0	661	781	810
Miami	1375	1763	1549	661	0	1426	1187
Dallas	967	1723	1548	781	1426	0	239
Houston	1087	1842	1627	810	1187	239	0



Motivation

```
double[][] distances = {  
    {0, 983, 787, 714, 1375, 967, 1087},  
    {983, 0, 214, 1102, 1763, 1723, 1842},  
    {787, 214, 0, 888, 1549, 1548, 1627},  
    {714, 1102, 888, 0, 661, 781, 810},  
    {1375, 1763, 1549, 661, 0, 1426, 1187},  
    {967, 1723, 1548, 781, 1426, 0, 239},  
    {1087, 1842, 1627, 810, 1187, 239, 0},  
};
```



Declare/Create Two-dimensional Arrays

```
// Declare array ref var  
dataType[][] refVar;
```

```
// Create array and assign its reference to variable  
refVar = new dataType[10][10];
```

```
// Combine declaration and creation in one statement  
dataType[][] refVar = new dataType[10][10];
```

```
// Alternative syntax  
dataType refVar[][] = new dataType[10][10];
```



Declaring Variables of Two-dimensional Arrays and Creating Two-dimensional Arrays

```
int[][] matrix = new int[10][10];
```

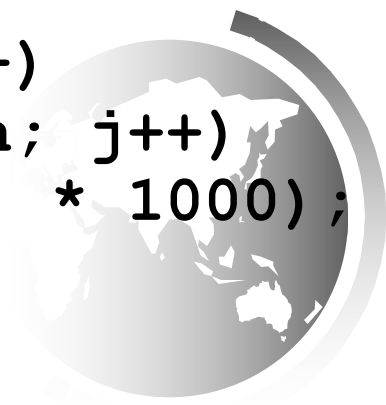
or

```
int matrix[][] = new int[10][10];
```

```
matrix[0][0] = 3;
```

```
for (int i = 0; i < matrix.length; i++)  
    for (int j = 0; j < matrix[i].length; j++)  
        matrix[i][j] = (int) (Math.random() * 1000);
```

```
double[][] x;
```



Two-dimensional Array Illustration

	[0]	[1]	[2]	[3]	[4]
[0]	0	0	0	0	0
[1]	0	0	0	0	0
[2]	0	0	0	0	0
[3]	0	0	0	0	0
[4]	0	0	0	0	0

```
matrix = new int[5][5];
```

(a)

matrix.length? 5

matrix[0].length? 5

	[0]	[1]	[2]	[3]	[4]
[0]	0	0	0	0	0
[1]	0	0	0	0	0
[2]	0	7	0	0	0
[3]	0	0	0	0	0
[4]	0	0	0	0	0

```
matrix[2][1] = 7;
```

(b)

	[0]	[1]	[2]
[0]	1	2	3
[1]	4	5	6
[2]	7	8	9
[3]	10	11	12

```
int[][] array = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9},  
    {10, 11, 12}  
};
```

(c)

array.length? 4

array[0].length? 3

Declaring, Creating, and Initializing Using Shorthand Notations

You can also use an array initializer to declare, create and initialize a two-dimensional array. For example,

```
int[][] array = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9},  
    {10, 11, 12}  
};
```

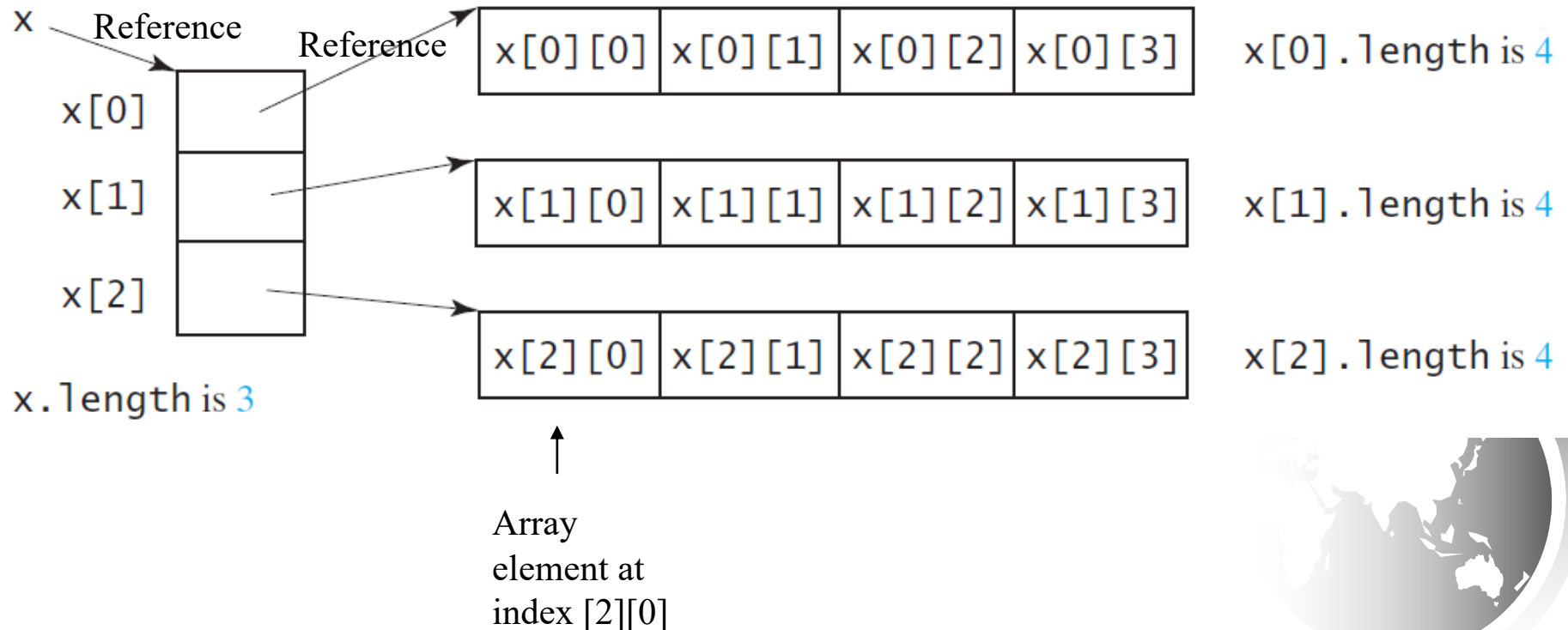
Same as

```
int[][] array = new int[4][3];  
array[0][0] = 1; array[0][1] = 2; array[0][2] = 3;  
array[1][0] = 4; array[1][1] = 5; array[1][2] = 6;  
array[2][0] = 7; array[2][1] = 8; array[2][2] = 9;  
array[3][0] = 10; array[3][1] = 11; array[3][2] = 12;
```


Lengths of Two-dimensional Arrays

```
int[][] x = new int[3][4];
```

Array reference variable



Lengths of Two-dimensional Arrays, cont.

```
int[][] array = {  
    {1, 2, 3},  
    {4, 5, 6},  
    {7, 8, 9},  
    {10, 11, 12}  
};
```

`array.length`

`array[0].length`

`array[1].length`

`array[2].length`

`array[3].length`

`array[4].length`

`ArrayIndexOutOfBoundsException`



Note

- ✦ The syntax `new int[5][]` for creating an array requires the first index to be specified.
- ✦ The syntax `new int[][]` would be wrong.



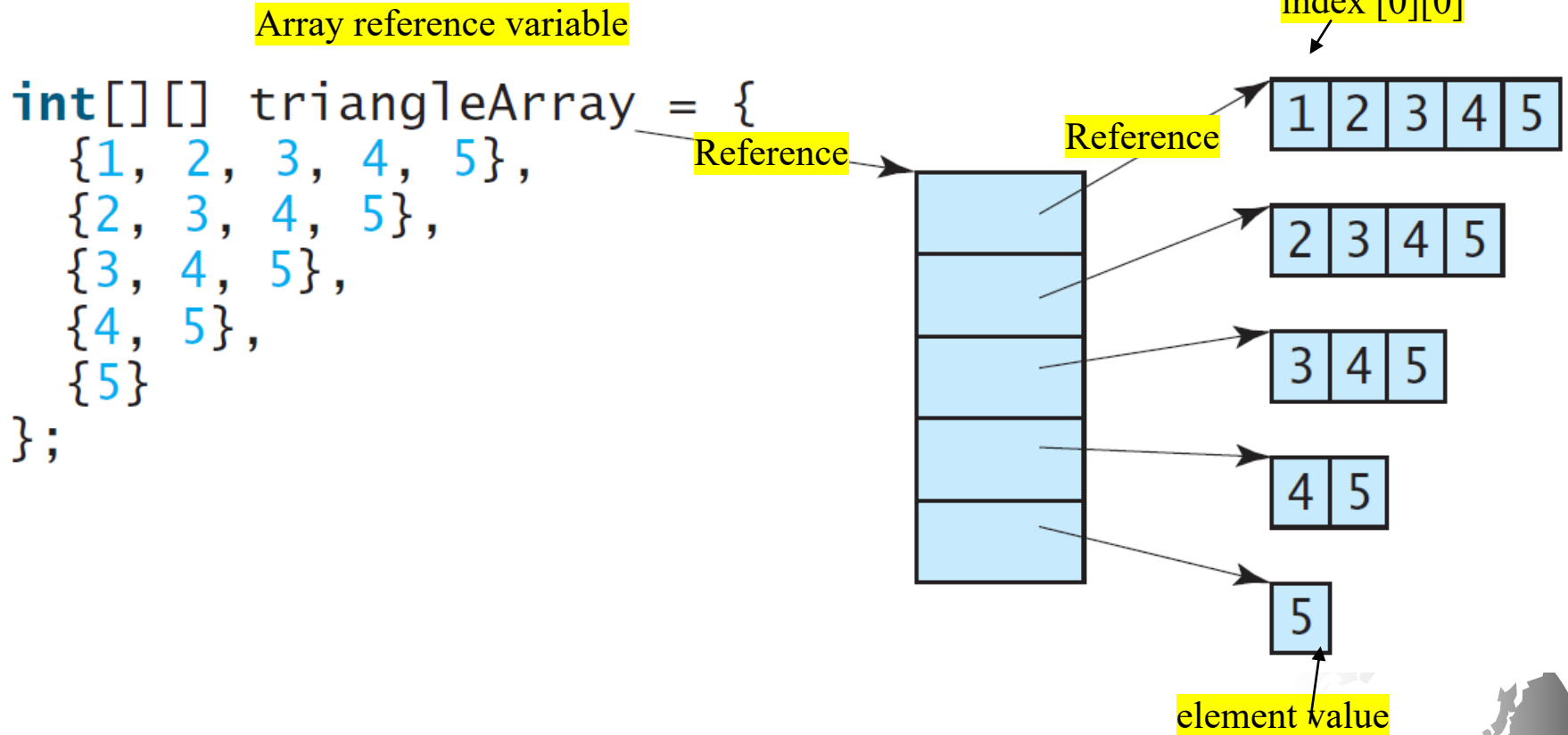
Ragged Arrays

Each row in a two-dimensional array is itself an array. So, the rows can have different lengths. Such an array is known as *a ragged array*. For example,

```
int[][] matrix = {  
    {1, 2, 3, 4, 5},  
    {2, 3, 4, 5},  
    {3, 4, 5},  
    {4, 5},  
    {5}  
};
```

```
matrix.length is 5  
matrix[0].length is 5  
matrix[1].length is 4  
matrix[2].length is 3  
matrix[3].length is 2  
matrix[4].length is 1
```

Ragged Arrays, cont.



array vs. array variable

- ♦ An array variable that appears to hold an array **actually contains a reference** to that array.
- ♦ Strictly speaking, an array variable and an array are different,
- ♦ But most of the time the distinction can be ignored.

Thus, it is all right to say, for simplicity, that myList is an array, instead of stating, at greater length, that myList is a variable that contains a reference to an array of ten double elements.

Processing Two-Dimensional Arrays

See the examples in the text.

1. (Initializing arrays with input values)
2. (Printing arrays)
3. (Summing all elements)
4. (Summing all elements by column)
5. (Which row has the largest sum)
6. (Finding the smallest index of the largest element)
7. (*Random shuffling*)



Processing Two-Dimensional Arrays

- ♦ *Nested for loops* are often used to process a two-dimensional array.
- ♦ Suppose an array matrix is created as follows:

```
int[][] matrix = new int[10][10];
```

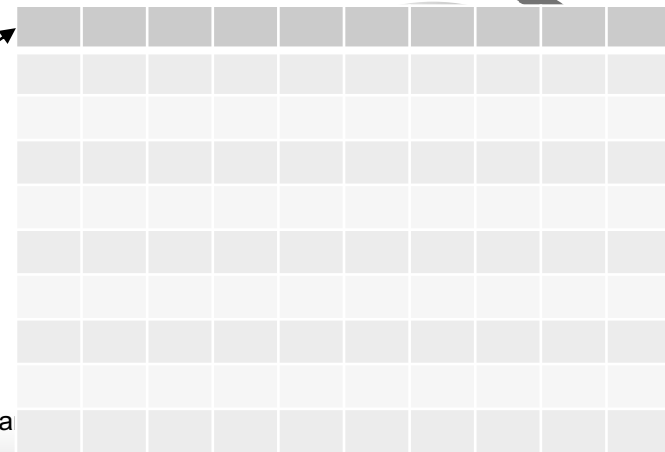


Initializing arrays with input values

Initializing arrays with input values. The following loop initializes the array with user input values:

```
java.util.Scanner input = new Scanner(System.in);
System.out.println("Enter " + matrix.length + " rows and " +
    matrix[0].length + " columns: ");
for (int row = 0; row < matrix.length; row++) {
    for (int column = 0; column < matrix[row].length; column++) {
        matrix[row][column] = input.nextInt();
    }
}
```

matrix

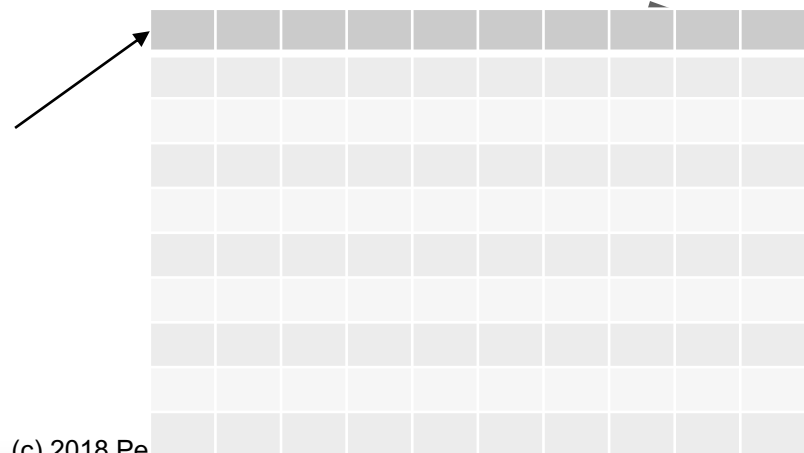


Initializing arrays with random values

Initializing arrays with random values. The following loop initializes the array with random values between 0 and 99:

```
for (int row = 0; row < matrix.length; row++) {  
    for (int column = 0; column < matrix[row].length; column++) {  
        matrix[row][column] = (int)(Math.random() * 100);  
    }  
}
```

matrix



Printing arrays

Printing arrays. To print a two-dimensional array, you have to print each element in the array using a loop like the following:

```
for (int row = 0; row < matrix.length; row++) {  
    for (int column = 0; column < matrix[row].length; column++) {  
        System.out.print(matrix[row][column] + " ");  
    }  
  
    System.out.println();  
}
```



Summing all elements

Summing all elements. Use a variable named total to store the sum.
Initially total is 0. Add each element in the array to total using a loop like this:

```
int total = 0;
for (int row = 0; row < matrix.length; row++) {
    for (int column = 0; column < matrix[row].length; column++) {
        total += matrix[row][column];
    }
}
```

1	1	2
2	2	4
1	3	1

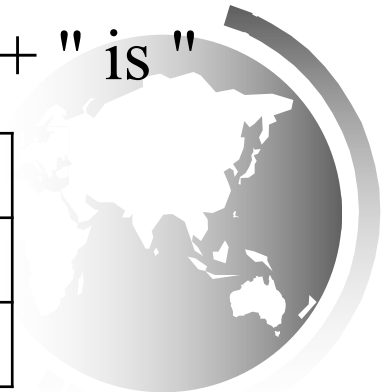


Summing elements by column

Summing elements by column. For each column, use a variable named `total` to store its sum. Add each element in the column to `total` using a loop like this:

```
for (int column = 0; column < matrix[0].length; column++) {  
    int total = 0;  
    for (int row = 0; row < matrix.length; row++)  
        total += matrix[row][column];  
    System.out.println("Sum for column " + column + " is "  
        + total);  
}
```

1	1	2
2	2	4
1	3	1



Summing elements by column

```
for (int column = 0; column < matrix[0].length; column++) {  
    int total = 0;  
    for (int row = 0; row < matrix.length; row++)  
        total += matrix[row][column];  
    System.out.println("Sum for column " + column + " is "  
        + total);  
}
```

What if we are using a ragged array?

What if `row[0].length < row[3].length`? HW



Summing elements by column

What if we are using a ragged array?

```
int[][] matrix = {  
    {1, 2, 3, 4, 5},  
    {2, 3, 4, 5},  
    {3, 4, 5},  
    {4, 5},  
    {5}  
};  
int count = matrix.length;  
for (int column = 0; column < matrix[0].length; column++) {  
    int total = 0;  
    for (int row = 0; row < count; row++){  
        total += matrix[row][column];  
    }  
    count--;  
    System.out.println("Sum for column " + column + " is " + total);  
}
```

What if `row[0].length < row[3].length`? HW



Find the row with the largest sum

```
int maxRow = 0;
int indexOfMaxRow = 0;
//Get sum of the first row in maxRow
for (int column = 0; column < matrix[0].length; column++)
{
    maxRow += matrix[0][column];
}
for (int row = 1; row < matrix.length; row++)
{
    int totalOfThisRow = 0;
    for (int column = 0; column < matrix[row].length; column++)
        totalOfThisRow += matrix[row][column];
    if (totalOfThisRow > maxRow)
    {
        maxRow = totalOfThisRow;
        indexOfMaxRow = row;
    }
}
System.out.println("Row" + indexOfMaxRow + " has the maximum sum of " + maxRow);
```




```

public static void main(String[] args) {
    int[][] arr = new int[3][];
    arr[0] = new int[2];
    arr[1] = new int[4];
    arr[2] = new int[3];
    for(int i=0; i<arr.length; i++)           //initialize the array
        for(int j=0; j<arr[i].length; j++){   arr[i][j]= i+1;    }
    PrintArray(arr);
}

```

```

The array row by row:
1 1
2 2 2 2
3 3 3
The array column by column:
1 2 3
1 2 3
2 3
2
BUILD SUCCESSFUL (total time: 1 second)

```

```

public static void PrintArray(int[][] arr){
    System.out.println("The array row by row:");    //print row by row
    for(int r=0; r<arr.length; r++){
        for(int c=0; c<arr[r].length; c++)
            System.out.print(arr[r][c]+" ");
        System.out.println();
    }
    System.out.println("The array column by column:"); //print column by column
    for(int c=0; c<arr[max_row].length; c++){ // assume we know the max row length
        for(int r=0; r<arr.length; r++){
            if(c>=arr[r].length)
                continue;
            System.out.print(arr[r][c]+" ");
        }
        System.out.println();
    }
}
}

```



Random shuffling

for each element `matrix[i][j]`, randomly generate indices `i1` and `j1` and swap `matrix[i][j]` with `matrix[i1][j1]`, as follows:

```
for (int i = 0; i < matrix.length; i++) {  
    for (int j = 0; j < matrix[i].length; j++) {  
        int i1 = (int)(Math.random() * matrix.length);  
        int j1 = (int)(Math.random() * matrix[i].length);  
        // Swap matrix[i][j] with matrix[i1][j1]  
        int temp = matrix[i][j];  
        matrix[i][j] = matrix[i1][j1];  
        matrix[i1][j1] = temp; }  
}
```



Passing Two-Dimensional Arrays to Methods

```
import java.util.Scanner;

public class PassTwoDimensionalArray {
    public static void main(String[] args) {
        int[][] m = getArray(); // Get an array

        // Display sum of elements
        System.out.println("\nSum of all elements is " + sum(m));
    }

    public static int[][] getArray() {
        // Create a Scanner
        Scanner input = new Scanner(System.in);

        // Enter array values
        int[][] m = new int[3][4];
        System.out.println("Enter " + m.length + " rows and "
            + m[0].length + " columns: ");
        for (int i = 0; i < m.length; i++)
            for (int j = 0; j < m[i].length; j++)
                m[i][j] = input.nextInt();

        return m;
    }

    public static int sum(int[][] m) {
        int total = 0;
        for (int row = 0; row < m.length; row++) {
            for (int column = 0; column < m[row].length; column++) {
                total += m[row][column];
            }
        }

        return total;
    }
}
```



Code Tracing

```
import java.util.Scanner;
```

```
public class PassTwoDimensionalArray {  
    public static void main(String[] args) {  
        int[][] m = getArray(); // Get an array  
  
        // Display sum of elements  
        System.out.println("\nSum of all elements is " + sum(m));  
    }  
}
```

The program starts the execution from the main method.



Code Tracing

```
import java.util.Scanner;

public class PassTwoDimensionalArray {
    public static void main(String[] args) {
        int[][] m = getArray(); // Get an array
        // Display sum of elements
        System.out.println("\nSum of all elements is " + sum(m));
    }

    public static int[][] getArray() {
        // Create a Scanner
        Scanner input = new Scanner(System.in);
        // Enter array values
        int[][] m = new int[3][4];
        System.out.println("Enter " + m.length + " rows and "
            + m[0].length + " columns: ");
        for (int i = 0; i < m.length; i++)
            for (int j = 0; j < m[i].length; j++)
                m[i][j] = input.nextInt();

        return m;
    }
}
```

The statement invokes `getArray()`.

The statement creates an input object for performing console input.



Code Tracing

```
import java.util.Scanner;

public class PassTwoDimensionalArray {
    public static void main(String[] args) {
        int[][] m = getArray(); // Get an array
        // Display sum of elements
        System.out.println("\nSum of all elements is " + sum(m));
    }
```

The statement invokes `getArray()`.

```
public static int[][] getArray() {
    // Create a Scanner
    Scanner input = new Scanner(System.in);

    // Enter array values
    int[][] m = new int[3][4];
    System.out.println("Enter " + m.length + " rows and "
        + m[0].length + " columns: ");
    for (int i = 0; i < m.length; i++)
        for (int j = 0; j < m[i].length; j++)
            m[i][j] = input.nextInt();

    return m;
}
```

This statement will
create a 3X4 array

0	0	0	0
0	0	0	0
0	0	0	0



Code Tracing

```
import java.util.Scanner;

public class PassTwoDimensionalArray {
    public static void main(String[] args) {
        int[][] m = getArray(); // Get an array
        // Display sum of elements
        System.out.println("\nSum of all elements is " + sum(m));
    }

    public static int[][] getArray() {
        // Create a Scanner
        Scanner input = new Scanner(System.in);

        // Enter array values
        int[][] m = new int[3][4];
        System.out.println("Enter " + m.length + " rows and "
            + m[0].length + " columns: ");
        for (int i = 0; i < m.length; i++)
            for (int j = 0; j < m[i].length; j++)
                m[i][j] = input.nextInt();

        return m;
    }
}
```

The statement invokes `getArray()`.

0	0	0	0
0	0	0	0
0	0	0	0

The code will output
a prompt
**Enter 3 rows and 4
columns :**

Code Tracing

```
import java.util.Scanner;

public class PassTwoDimensionalArray {
    public static void main(String[] args) {
        int[][] m = getArray(); // Get an array
        // Display sum of elements
        System.out.println("\nSum of all elements is " + sum(m));
    }

    public static int[][] getArray() {
        // Create a Scanner
        Scanner input = new Scanner(System.in);

        // Enter array values
        int[][] m = new int[3][4];
        System.out.println("Enter " + m.length + " rows and "
            + m[0].length + " columns: ");
        for (int i = 0; i < m.length; i++)
            for (int j = 0; j < m[i].length; j++)
                m[i][j] = input.nextInt();

        return m;
    }
}
```

0	0	0	0
0	0	0	0
0	0	0	0

Reading from the user
The user need to enter how
many values ?



Code Tracing

```
import java.util.Scanner;

public class PassTwoDimensionalArray {
    public static void main(String[] args) {
        int[][] m = getArray(); // Get an array
        // Display sum of elements
        System.out.println("\nSum of all elements is " + sum(m));
    }

    public static int[][] getArray() {
        // Create a Scanner
        Scanner input = new Scanner(System.in);

        // Enter array values
        int[][] m = new int[3][4];
        System.out.println("Enter " + m.length + " rows and "
            + m[0].length + " columns: ");
        for (int i = 0; i < m.length; i++)
            for (int j = 0; j < m[i].length; j++)
                m[i][j] = input.nextInt();

        return m;
    }
}
```

The statement invokes `getArray()`.

1	1	1	1
2	2	2	2
3	3	3	3

If the user entered
1 1 1 1 2 2 2 2 3 3 3 3

Code Tracing

```
import java.util.Scanner;

public class PassTwoDimensionalArray {
    public static void main(String[] args) {
        int[][] m = getArray(); // Get an array
        // Display sum of elements
        System.out.println("\nSum of all elements is " + sum(m));
    }

    public static int[][] getArray() {
        // Create a Scanner
        Scanner input = new Scanner(System.in);

        // Enter array values
        int[][] m = new int[3][4];
        System.out.println("Enter " + m.length + " rows and "
            + m[0].length + " columns: ");
        for (int i = 0; i < m.length; i++)
            for (int j = 0; j < m[i].length; j++)
                m[i][j] = input.nextInt();

        return m;
    }
}
```

The statement invokes `getArray()`.

The array will be returned to the main



Code Tracing

```
import java.util.Scanner;

public class PassTwoDimensionalArray {
    public static void main(String[] args) {
        int[][] m = getArray(); // Get an array

        // Display sum of elements
        System.out.println("\nSum of all elements is " + sum(m));
    }

    public static int[][] getArray() {
        // Create a Scanner
        Scanner input = new Scanner(System.in);

        // Enter array values
        int[][] m = new int[3][4];
        System.out.println("Enter " + m.length + " rows and "
            + m[0].length + " columns: ");
        for (int i = 0; i < m.length; i++)
            for (int j = 0; j < m[i].length; j++)
                m[i][j] = input.nextInt();

        return m;
    }
}
```



1	1	1	1
2	2	2	2
3	3	3	3



Code Tracing

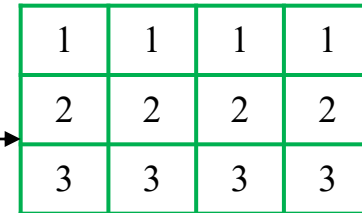
```
import java.util.Scanner;
```

```
public class PassTwoDimensionalArray {  
    public static void main(String[] args) {  
        int[][] m = getArray(); // Get an array
```

```
        // Display sum of elements
```

```
        System.out.println("\nSum of all elements is " + sum(m));
```

```
    }
```



1	1	1	1
2	2	2	2
3	3	3	3

Invoking sum method

```
public static int sum(int[][] m) {
```

```
    int total = 0;
```

```
    for (int row = 0; row < m.length; row++) {
```

```
        for (int column = 0; column < m[row].length; column++) {
```

```
            total += m[row][column];
```

```
        }
```

```
    }
```

```
    return total;
```

```
}
```



Problem: Grading Multiple-Choice Test

Students' answer

	0	1	2	3	4	5	6	7	8	9
Student 0	A	B	A	C	C	D	E	E	A	D
Student 1	D	B	A	B	C	A	E	E	A	D
Student 2	E	D	D	A	C	B	E	E	A	D
Student 3	C	B	A	E	D	C	E	E	A	D
Student 4	A	B	D	C	C	D	E	E	A	D
Student 5	B	B	E	C	C	D	E	E	A	D
Student 6	B	B	A	C	C	D	E	E	A	D
Student 7	E	B	E	C	C	D	E	E	A	D

Objective: write a program that grades multiple-choice test.

Key to the Questions:

	0	1	2	3	4	5	6	7	8	9
Key	D	B	D	C	C	D	A	E	A	D

PassTwoDimensionalArray

Run

```

public class GradeExam {
public static void main(String args[]) {
// Students' answers to the questions
char[][] answers = { {'A', 'B', 'A', 'C', 'C', 'D', 'E', 'E', 'A', 'D'}, {'D', 'B',
'A', 'B', 'C', 'A', 'E', 'E', 'A', 'D'}, {'E', 'D', 'D', 'A', 'C', 'B', 'E', 'E', 'A',
'D'}, {'C', 'B', 'A', 'E', 'D', 'C', 'E', 'E', 'A', 'D'}, {'A', 'B', 'D', 'C', 'C', 'D',
'E', 'E', 'A', 'D'}, {'B', 'B', 'E', 'C', 'C', 'D', 'E', 'E', 'A', 'D'}, {'B', 'B', 'A',
'C', 'C', 'D', 'E', 'E', 'A', 'D'}, {'E', 'B', 'E', 'C', 'C', 'D', 'E', 'E', 'A', 'D'}};

// Key to the questions
char[] keys = {'D', 'B', 'D', 'C', 'C', 'D', 'A', 'E', 'A', 'D'};
// Grade all answers
for (int i = 0; i < answers.length; i++) {
// Grade one student
    int correctCount = 0;
    for (int j = 0; j < answers[i].length; j++) {
        if (answers[i][j] == keys[j])
            correctCount++;
    }
    System.out.println("Student " + i + "'s correct count is " + correctCount);
}
}

```



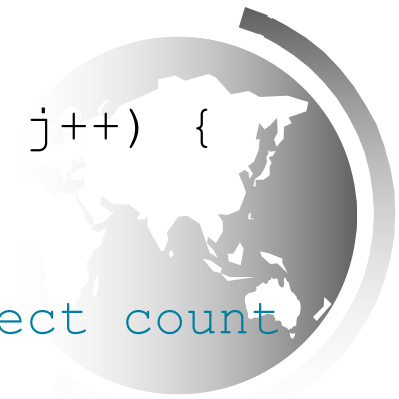
Students' answer

	0	1	2	3	4	5	6	7	8	9
Student 0	A	B	A	C	C	D	E	E	A	D
Student 1	D	B	A	B	C	A	E	E	A	D
Student 2	E	D	D	A	C	B	E	E	A	D
Student 3	C	B	A	E	D	C	E	E	A	D
Student 4	A	B	D	C	C	D	E	E	A	D
Student 5	B	B	E	C	C	D	E	E	A	D
Student 6	B	B	A	C	C	D	E	E	A	D
Student 7	E	B	E	C	C	D	E	E	A	D

Key to the Questions:

	0	1	2	3	4	5	6	7	8	9
Key	D	B	D	C	C	D	A	E	A	D

```
for (int i = 0; i < answers.length; i++) {  
    // Grade one student  
    int correctCount = 0;  
    for (int j = 0; j < answers[i].length; j++) {  
        if (answers[i][j] == keys[j])  
            correctCount++;  
    }  
    System.out.println("Student " + i + "'s correct count  
is " + correctCount);  
}
```



Answer

- ◆ Student 0's correct count is 7
- ◆ Student 1's correct count is 6
- ◆ Student 2's correct count is 5
- ◆ Student 3's correct count is 4
- ◆ Student 4's correct count is 8
- ◆ Student 5's correct count is 7
- ◆ Student 6's correct count is 7
- ◆ Student 7's correct count is 7



Multidimensional Arrays

- Occasionally, you will need to represent n-dimensional data structures.
- In Java, you can create n-dimensional arrays for any integer n.
- The way to declare two-dimensional array variables and create two-dimensional arrays can be generalized to declare n-dimensional array variables and create n-dimensional arrays for $n \geq 3$.



Multidimensional Arrays

- ◆ A multidimensional array is actually an array in which each element is another array.
- ◆ A three-dimensional array consists of an array of two-dimensional arrays. A two-dimensional array consists of an array of one-dimensional arrays.



Multidimensional Arrays

- ✦ For example, you may use a three-dimensional array to store exam scores for a class of **six students** with **five exams**, and each exam has **two parts** (multiple-choice and essay).
- ✦ The following syntax declares a three-dimensional array variable `scores`, creates an array, and assigns its reference to `scores`.
- ✦ `double[][][] scores = new double[6][5][2];`

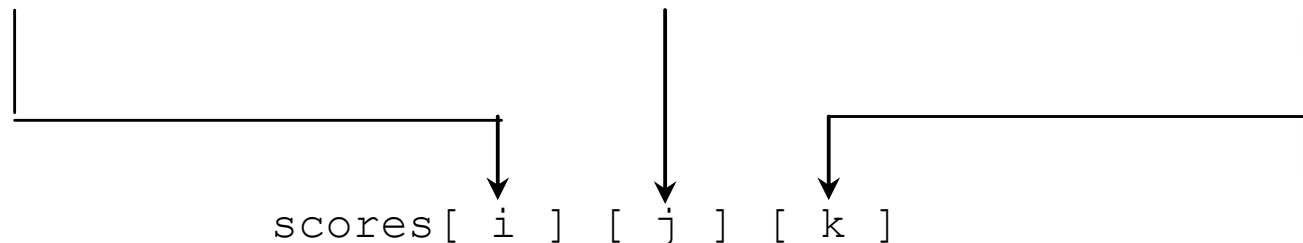
Multidimensional Arrays

```
double[][][] scores = {  
    {{7.5, 20.5}, {9.0, 22.5}, {15, 33.5}, {13, 21.5}, {15, 2.5}},  
    {{4.5, 21.5}, {9.0, 22.5}, {15, 34.5}, {12, 20.5}, {14, 9.5}},  
    {{6.5, 30.5}, {9.4, 10.5}, {11, 33.5}, {11, 23.5}, {10, 2.5}},  
    {{6.5, 23.5}, {9.4, 32.5}, {13, 34.5}, {11, 20.5}, {16, 7.5}},  
    {{8.5, 26.5}, {9.4, 52.5}, {13, 36.5}, {13, 24.5}, {16, 2.5}},  
    {{9.5, 20.5}, {9.4, 42.5}, {13, 31.5}, {12, 20.5}, {16, 6.5}}  
};
```

Which student

Which exam

Multiple-choice or essay



Multidimensional Arrays

```
double[][][] scores = {  
    {{7.5, 20.5}, {9.0, 22.5}, {15, 33.5}, {13, 21.5}, {15, 2.5}},  
    {{4.5, 21.5}, {9.0, 22.5}, {15, 34.5}, {12, 20.5}, {14, 9.5}},  
    {{6.5, 30.5}, {9.4, 10.5}, {11, 33.5}, {11, 23.5}, {10, 2.5}},  
    {{6.5, 23.5}, {9.4, 32.5}, {13, 34.5}, {11, 20.5}, {16, 7.5}},  
    {{8.5, 26.5}, {9.4, 52.5}, {13, 36.5}, {13, 24.5}, {16, 2.5}},  
    {{9.5, 20.5}, {9.4, 42.5}, {13, 31.5}, {12, 20.5}, {16, 6.5}}  
};
```

scores[0][1][0] refers to the
multiple-choice score for the first
student's second exam,

which is 9.0.

scores[0][1][1] refers to the essay
score for the first student's second
exam,

which is 22.5.

Multidimensional Arrays

suppose `x = new int[2][2][5]`

`x[0]` and `x[1]` are two-dimensional arrays.

`x[0][0]`, `x[0][1]`, `x[1][0]`, and `x[1][1]` are one-dimensional arrays and each contains five elements.

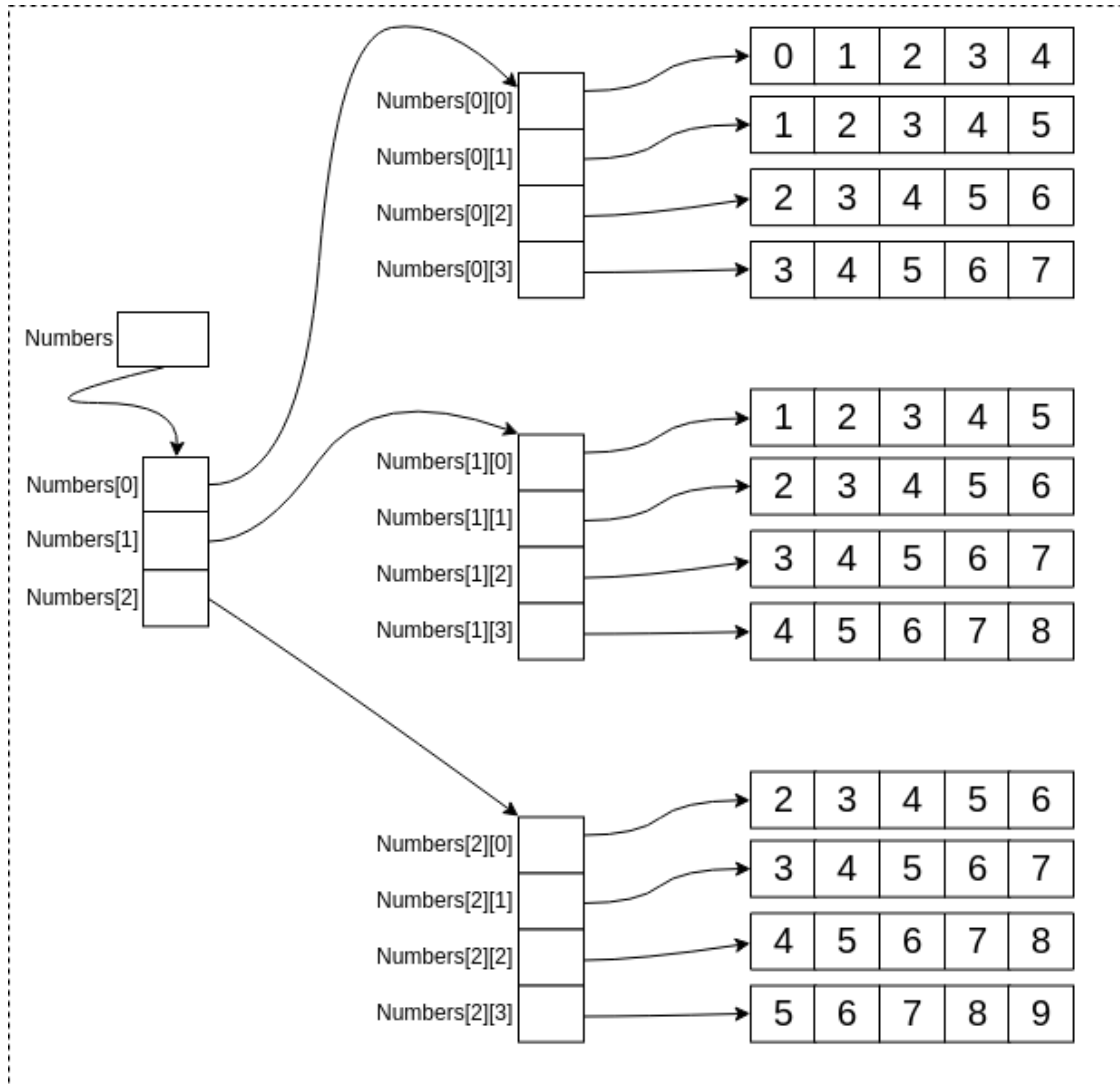
`x.length = 2`

`x[0].length = x[1].length = 2`

`x[0][0].length = x[0][1].length = x[1][0].length =
x[1][1].length = 5`



Multidimensional Arrays



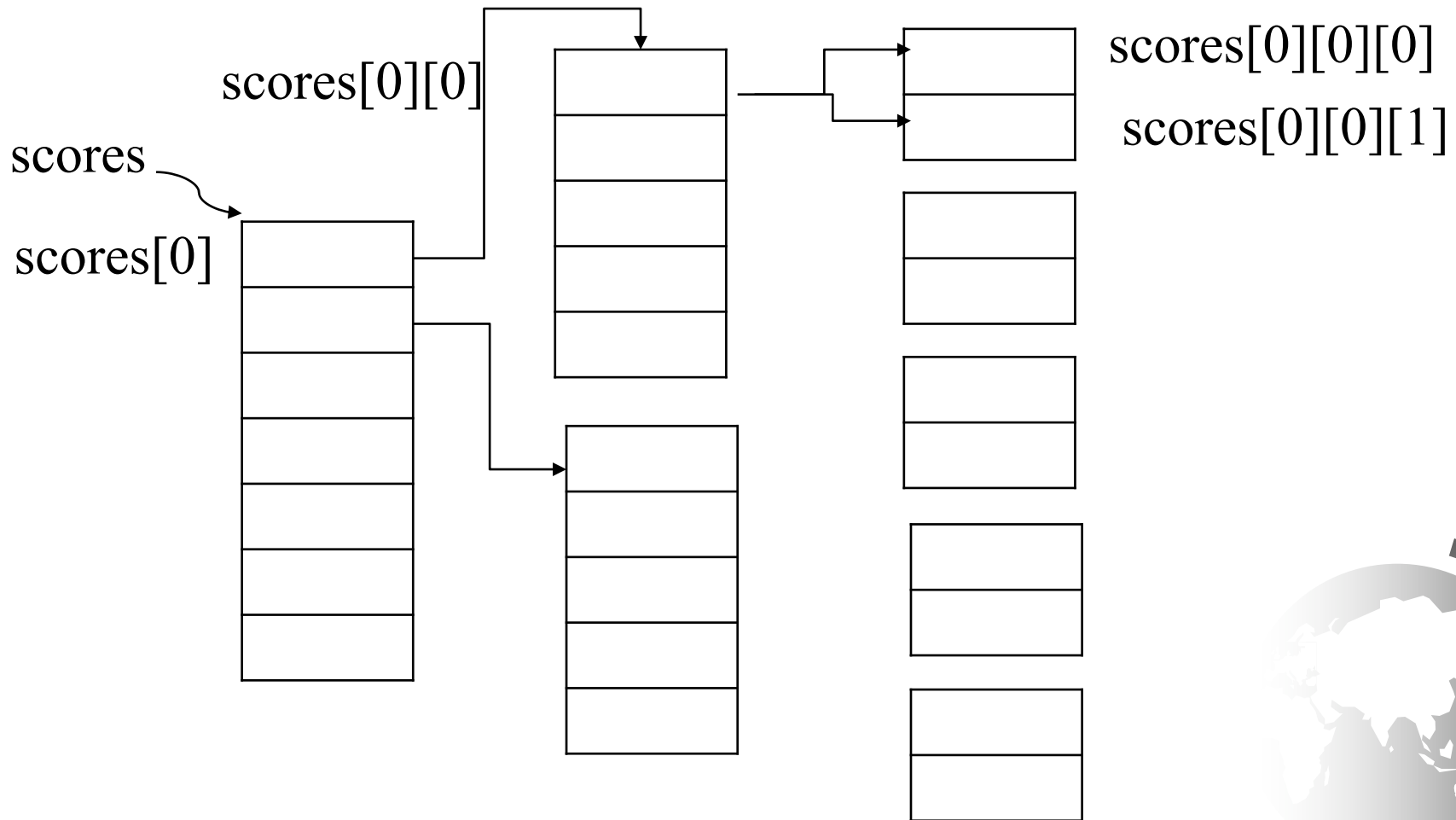
Problem: Calculating Total Scores

- ♦ Objective: write a program that calculates the total score for students in a class. Suppose the scores are stored in a three-dimensional array named scores. The first index in scores refers to a **student**, the second refers to an **exam**, and the third refers to **the part of the exam**. Suppose there are **7 students**, **5 exams**, and each exam has **two parts**--the multiple-choice part and the programming part. So, scores[i][j][0] represents the score on the multiple-choice part for the *i*'s student on the *j*'s exam. Your program displays the total score for each student.




```
double[][][] scores = {  
    {{7.5, 20.5}, {9.0, 22.5}, {15, 33.5}, {13, 21.5}, {15, 2.5}},  
    {{4.5, 21.5}, {9.0, 22.5}, {15, 34.5}, {12, 20.5}, {14, 9.5}},  
    {{6.5, 30.5}, {9.4, 10.5}, {11, 33.5}, {11, 23.5}, {10, 2.5}},  
    {{6.5, 23.5}, {9.4, 32.5}, {13, 34.5}, {11, 20.5}, {16, 7.5}},  
    {{8.5, 26.5}, {9.4, 52.5}, {13, 36.5}, {13, 24.5}, {16, 2.5}},  
    {{9.5, 20.5}, {9.4, 42.5}, {13, 31.5}, {12, 20.5}, {16, 6.5}},  
    {{1.5, 29.5}, {6.4, 22.5}, {14, 30.5}, {10, 30.5}, {16, 6.0}}};
```





```
for (int i = 0; i < scores.length; i++) {  
    double totalScore = 0;  
    for (int j = 0; j < scores[i].length; j++)  
        for (int k = 0; k < scores[i][j].length; k++)  
            totalScore += scores[i][j][k];  
}
```

```
System.out.println("Student " + i + "'s  
score is " + totalScore); }
```



```

public class TotalScore {
    public static void main(String args[]) {

        double[][][] scores = {
            {{7.5, 20.5}, {9.0, 22.5}, {15, 33.5}, {13, 21.5}, {15, 2.5}},
            {{4.5, 21.5}, {9.0, 22.5}, {15, 34.5}, {12, 20.5}, {14, 9.5}},
            {{6.5, 30.5}, {9.4, 10.5}, {11, 33.5}, {11, 23.5}, {10, 2.5}},
            {{6.5, 23.5}, {9.4, 32.5}, {13, 34.5}, {11, 20.5}, {16, 7.5}},
            {{8.5, 26.5}, {9.4, 52.5}, {13, 36.5}, {13, 24.5}, {16, 2.5}},
            {{9.5, 20.5}, {9.4, 42.5}, {13, 31.5}, {12, 20.5}, {16, 6.5}},
            {{1.5, 29.5}, {6.4, 22.5}, {14, 30.5}, {10, 30.5}, {16, 6.0}}};
    
```

```

// Calculate and display total score for each student
for (int i = 0; i < scores.length; i++) {
    double totalScore = 0;
    for (int j = 0; j < scores[i].length; j++)
        for (int k = 0; k < scores[i][j].length; k++)
            totalScore += scores[i][j][k];
    System.out.println("Student " + i + "'s score is " + totalScore);
}
}

```



Answer

- ◆ Student 0's score is 160.0
- ◆ Student 1's score is 163.0
- ◆ Student 2's score is 148.4
- ◆ Student 3's score is 174.4
- ◆ Student 4's score is 202.4
- ◆ Student 5's score is 181.4
- ◆ Student 6's score is 166.9



Problem: Weather Information

- ◆ Suppose a meteorology station records the **temperature and humidity** at each hour of every day and stores the data for the past ten days in a text file named `weather.txt`. Each line of the file consists of four numbers that indicate **the day**, **hour**, **temperature**, and **humidity**. Your task is to write a program that calculates the average daily temperature and humidity for the 10 days.

```
1 1 76.4 0.92
1 2 77.7 0.93
...
10 23 97.7 0.71
10 24 98.7 0.74
```

(a)

```
10 24 98.7 0.74
1 2 77.7 0.93
...
10 23 97.7 0.71
1 1 76.4 0.92
```

(b)

Weather

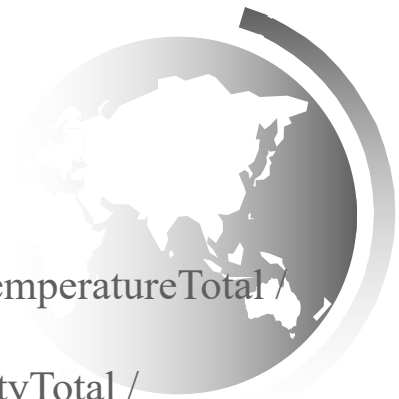
Run

```

public class Weather {
public static void main(String[] args) {

final int NUMBER_OF_DAYS = 10;
final int NUMBER_OF_HOURS = 24;
double[][][] data = new double[NUMBER_OF_DAYS][NUMBER_OF_HOURS][2];
Scanner input = new Scanner(System.in);
// Read input using input redirection from a file
for (int k = 0; k < NUMBER_OF_DAYS * NUMBER_OF_HOURS; k++) {
    int day = input.nextInt();
    int hour = input.nextInt();
    double temperature = input.nextDouble();
    double humidity = input.nextDouble();
    data[day - 1][hour - 1][0] = temperature;
    data[day - 1][hour - 1][1] = humidity;
}
// Find the average daily temperature and humidity
for (int i = 0; i < NUMBER_OF_DAYS; i++) {
    double dailyTemperatureTotal = 0,
    dailyHumidityTotal = 0;
    for (int j = 0; j < NUMBER_OF_HOURS; j++) {
        dailyTemperatureTotal += data[i][j][0];
        dailyHumidityTotal += data[i][j][1];
    }
    System.out.println("Day " + i + "'s average temperature is " + dailyTemperatureTotal /
    NUMBER_OF_HOURS);
    System.out.println("Day " + i + "'s average humidity is " + dailyHumidityTotal /
    NUMBER_OF_HOURS);
}
}
}
}

```



```
import java.util.Scanner;
```

```
public class Weather {  
public static void main(String[] args) {
```

```
final int NUMBER_OF_DAYS = 10;  
final int NUMBER_OF_HOURS = 24;
```

```
double[][][] data = new  
double[NUMBER_OF_DAYS][NUMBER_OF_HOURS][2];
```




```
Scanner input = new Scanner(System.in);

// Read input using input redirection from a
file
for (int k = 0; k < NUMBER_OF_DAYS *
NUMBER_OF_HOURS; k++)
{ int day = input.nextInt();
int hour = input.nextInt();
double temperature = input.nextDouble();
double humidity = input.nextDouble();
data[day - 1][hour - 1][0] = temperature;
data[day - 1][hour - 1][1] = humidity; }
```



```
// Find the average daily temperature and
humidity
for (int i = 0; i < NUMBER_OF_DAYS; i++) {
    double dailyTemperatureTotal = 0,
    dailyHumidityTotal = 0;

    for (int j = 0; j < NUMBER_OF_HOURS; j++)
    { dailyTemperatureTotal += data[i][j][0];
      dailyHumidityTotal += data[i][j][1]; }
}
```



```
// Display result
System.out.println("Day " + (i + 1) +
    "'s average temperature is " +
    dailyTemperatureTotal /
    NUMBER_OF_HOURS);
System.out.println("Day " + (i + 1) +
    "'s average humidity is " +
    dailyHumidityTotal / NUMBER_OF_HOURS);
```



Problem: Guessing Birthday

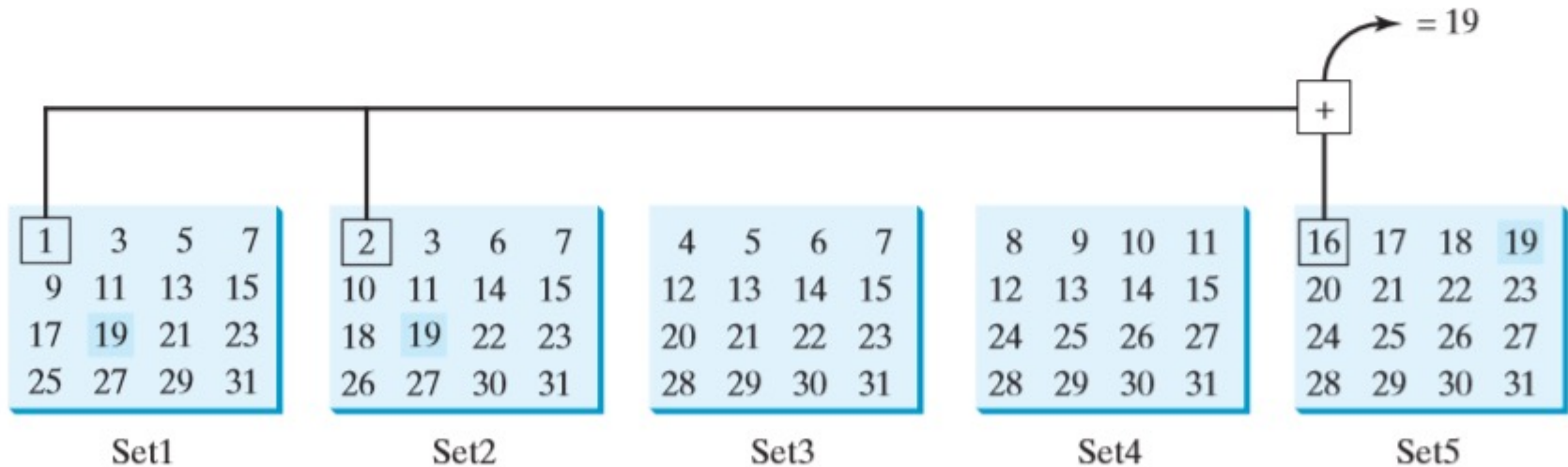
Listing 4.3, `GuessBirthday.java`, gives a program that guesses a birthday. The program can be simplified by storing the numbers in five sets in a three-dimensional array, and it prompts the user for the answers using a loop.

A graphic representing an IDE snippet. It features a green rectangular box containing the text 'GuessBirthdayUsingArray' and a blue rectangular button to its right with the text 'Run'. In the background, there is a faint, stylized globe showing the continents of North and South America.

GuessBirthdayUsingArray

Run

Let's Play ☺



```
int day = 0; // Day to be determined
int answer;
int[][][] dates = { { { 1, 3, 5, 7}, {
9, 11, 13, 15}, {17, 19, 21, 23}, {25,
27, 29, 31}}, { { 2, 3, 6, 7}, {10, 11,
14, 15}, {18, 19, 22, 23}, {26, 27,
30, 31}}, { { 4, 5, 6, 7}, {12, 13, 14,
15}, {20, 21, 22, 23}, {28, 29, 30,
31}}, { { 8, 9, 10, 11}, {12, 13, 14,
15}, {24, 25, 26, 27}, {28, 29, 30,
31}}, { {16, 17, 18, 19}, {20, 21, 22,
23}, {24, 25, 26, 27}, {28, 29, 30,
31}} };
```



```
// Create a Scanner
Scanner input = new Scanner(System.in);
for (int i = 0; i < 5; i++) {
    System.out.println("Is your birthday in
Set" + (i + 1) + "?");
    for (int j = 0; j < 4; j++) {
        for (int k = 0; k < 4; k++)
            System.out.printf("%4d", dates[i][j][k]);
        System.out.println();
    }
    System.out.print("\nEnter 0 for No and 1
for Yes: ");
    int answer = input.nextInt();
    if (answer == 1)
        day += dates[i][0][0];
    System.out.println("Your birthday is " +
day);
}
```

